

Министерство образования и науки Российской Федерации
Санкт-Петербургский Политехнический Университет Петра Великого

—
Институт компьютерных наук и кибербезопасности

ЛАБОРАТОРНАЯ РАБОТА № 2
«МЕХАНИЗМЫ СЕТЕВОГО ВЗАИМОДЕЙСТВИЯ»
по дисциплине «Операционные системы»

Выполнил
студент гр.5151001/40001

<подпись>

Волошкевич М.А.

Преподаватель /
ассистент

<подпись>

Гавва Г. Д.

Санкт-Петербург
2026г.

1. Цель работы.

Цель работы — изучить программный интерфейс сетевых сокетов, получить навыки организации взаимодействия программ при помощи протоколов Internet и разработки прикладных сетевых сервисов.

2. Постановка задачи

1. Задача 1

- Необходимо реализовать сетевую программу (tcpclient.cpp), которая считывала бы все эти сообщения из текстового файла и передавала их на удаленный сервер, прослушивающий TCP порт. Протокол взаимодействия клиента и сервера указан в вашем варианте задания. Адрес и порт сервера, а также имя входного файла указываются как аргументы командной строки при запуске клиента, например: tcpclient 192.168.50.7:9000 file1.txt
где tcpclient - имя исполняемого бинарного файла клиента (получен путем компиляции tcpclient.cpp),
192.168.50.7 - IPv4 адрес сервера, 9000 - прослушиваемый сервером TCP-порт, file1.txt - файл с сообщениями.
- Необходимо реализовать на языке C/C++ сервер tcpserver.cpp, который можно было бы использовать вместо эмулятора (tcpserveremul.rb). Сервер должен прослушивать TCP-порт, принимать входящие подключения от клиентов, а затем - принимать сообщения, передаваемые клиентами. На каждое полученное сообщение сервер отправляет клиенту подтверждение «ok», согласно спецификации протокола. Все полученные сообщения сервер распечатывает в файл msg.txt. Каждое сообщение в msg.txt предваряется данными о клиенте: IP-адрес, двоеточие, порт клиента, пробел.
Далее следует сообщение, которое распечатывается в таком же виде, как и было во входном файле клиента. Если от какого-либо клиента пришло сообщение с текстом «stop», то сервер, после отправки этому клиенту подтверждения «ok», закрывает все открытые соединения (отключает всех клиентов) и завершает работу. Номер прослушиваемого порта передается серверу при запуске как первый аргумент командной строки, например: tcpserver 9000 где tcpserver -

имя бинарного файла программы, 9000 - номер TCP порта для прослушивания.

2. Задача 2

- Реализуйте программу `udpclient.cpp`, взаимодействующую с удаленным сервером по протоколу UDP. Программа запускается аналогично `tcpclient`, с указанием IP-адреса и порта удаленного сервера (но уже UDP-порта) и имени входного файла с сообщениями:
`udpclient 192.168.50.7:8700 file1.txt`
- Реализуйте `udpserver.cpp`, который можно было бы использовать вместо Ruby-сервера-эмулятора.

3. Ход работы

1. Описание алгоритма работы TCP-клиента

TCP-клиент при запуске разбирает аргументы командной строки, извлекая IPv4-адрес сервера, порт и имя входного файла. Далее создаётся TCP-сокеты, и клиент устанавливает соединение с сервером, делая до 10 попыток с интервалом 100 мс. После успешного подключения клиент отправляет серверу маркер начала сессии — строку «put». Затем программа построчно читает входной файл, парсит дату, два временных штампа и текст сообщения, формирует бинарный пакет фиксированного формата (4 байта номер, 4 байта дата, 4+4 байта время, 4 байта длина, N байт текст) и отправляет его серверу. После каждой отправки клиент ожидает от сервера подтверждение «ok» перед отправкой следующего пакета. При получении сообщения «stop» или достижении конца файла клиент закрывает соединение и завершает работу.

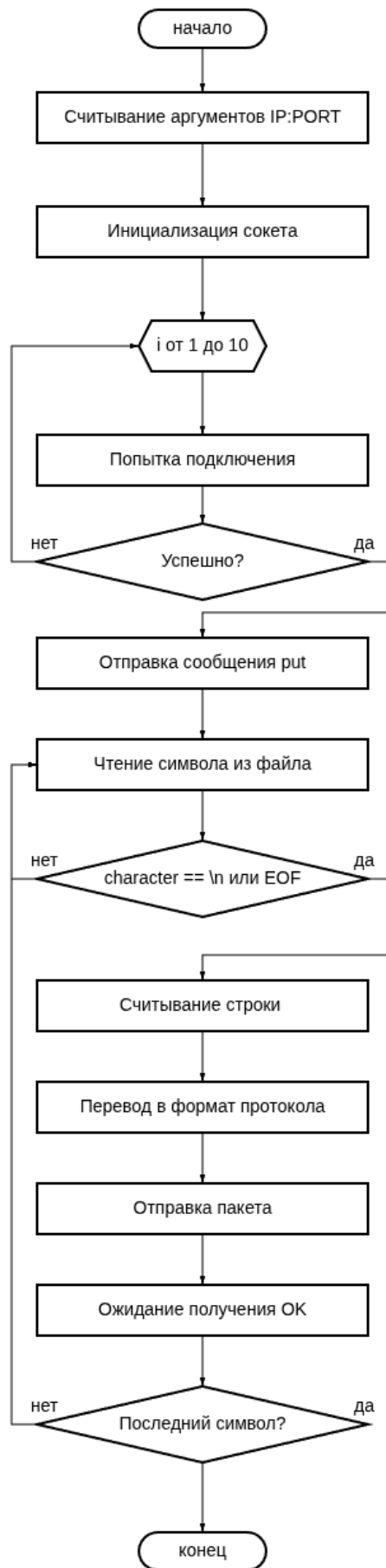


Рис. 1. Блок-схема TCP-клиента

Функция	Описание
<code>int send_request(int s, unsigned char* request, int size)</code>	Отправляет сформированный бинарный пакет на сервер, гарантируя полную передачу всех байт в цикле
<code>void pack_message_data(uint32_t msg_num, uint32_t date, uint32_t time1, uint32_t time2, char* message, unsigned char* buffer)</code>	Упаковывает поля сообщения в бинарный буфер в сетевом порядке байт (big-endian)
<code>int recv_ok(int s)</code>	Блокирующий приём двухбайтного подтверждения «ok» от сервера; возвращает ошибку при нарушении протокола
<code>unsigned int create_response(int s, FILE* f)</code>	Читает строки из файла, парсит дату и время, формирует пакет и последовательно отправляет каждое сообщение, ожидая «ok»
<code>int tok_numbers(char* src, const char* delim, bool is_reverse)</code>	Разбивает строку по разделителю и собирает числовое представление даты или времени в формате YYYYMMDD или HHMMSS
<code>int main(int argc, char* argv[])</code>	Разбирает аргументы командной строки, инициализирует сокет, устанавливает соединение с сервером и запускает цикл отправки сообщений

Таблица 1. Функция программы ТСП-клиента

1. Описание алгоритма работы TSP-сервера

TSP-сервер при запуске создаёт слушающий сокет, привязывает его к указанному порту и переводит в неблокирующий режим. Обработка нескольких клиентов одновременно реализована через системный вызов `poll`, отслеживающий события на всех активных дескрипторах. При поступлении нового подключения сервер принимает его функцией `accept` и добавляет клиента в массив, выделяя для него динамический буфер. От каждого клиента сервер сначала ожидает маркер «put», после чего начинает принимать бинарные пакеты с сообщениями. Каждый принятый пакет декодируется: извлекаются номер сообщения, дата, два временных штампа и текст, которые записываются в файл `msg.txt` с указанием IP-адреса и порта клиента. На каждое корректное сообщение сервер отправляет клиенту подтверждение «ok», а при получении сообщения «stop» закрывает все соединения и завершает работу.

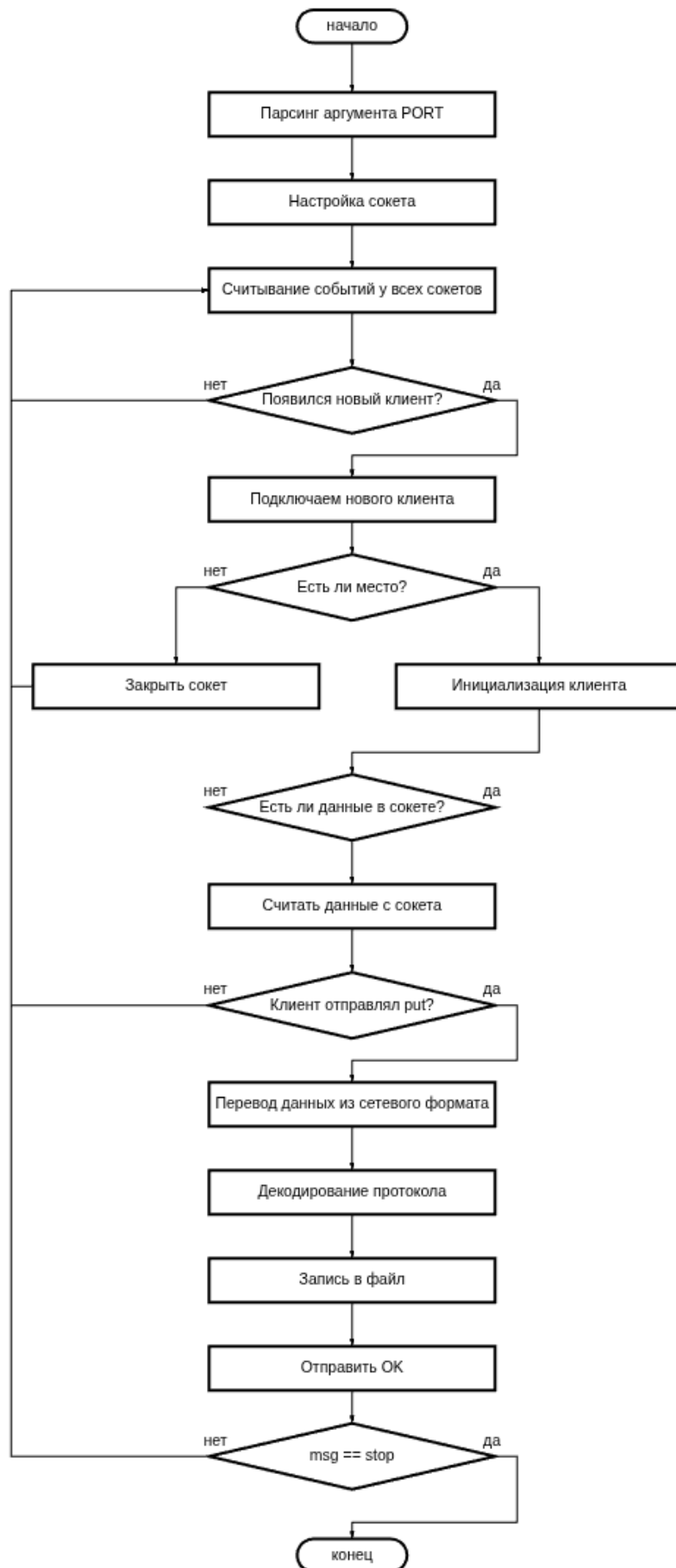


Рис. 2. Блок-схема TCP-сервера

Функция	Описание
<code>void set_nonblocking(socket_t s)</code>	Переводит сокет в неблокирующий режим с помощью <code>fcntl</code> (POSIX) или <code>ioctlsocket</code> (Windows)
<code>uint32_t read_u32(unsigned char* p)</code>	Читает 4 байта из буфера и преобразует их из сетевого порядка байт в хостовый
<code>void write_msg(FILE* f, struct sockaddr_in* addr, uint32_t date, uint32_t t1, uint32_t t2, char* msg)</code>	Форматирует и записывает принятое сообщение в файл <code>msg.txt</code> с указанием адреса, порта клиента, даты, времени и текста
<code>void close_socket(socket_t s)</code>	Кросс-платформенное закрытие сокета: <code>closesocket</code> на Windows, <code>close</code> на POSIX
<code>int main(int argc, char* argv[])</code>	Инициализирует сервер, запускает цикл <code>poll</code> для приёма подключений и обработки входящих сообщений от всех клиентов

Таблица 2. Функция программы TCP-сервера

1. Описание алгоритма работы UDP-клиента

UDP-клиент создаёт датаграммный сокет и без установки соединения начинает отправку пакетов на указанный IP-адрес и порт сервера. Каждое сообщение из входного файла упаковывается в бинарный датаграмм того же формата, что и в TCP-версии: номер, дата, время, длина и текст сообщения. После первичной отправки всех пакетов клиент переходит в цикл повторных передач, используя массив `on_server` для отслеживания того, какие сообщения уже подтверждены сервером. Для получения подтверждений применяется функция `select` с таймаутом 100 мс: сервер возвращает список номеров принятых сообщений в виде массива `uint32_t`. Клиент повторно отправляет только те сообщения, которые ещё не попали в список подтверждённых. Цикл завершается, когда число подтверждённых сообщений достигает ожидаемого количества (или 20, если сообщений больше 20).

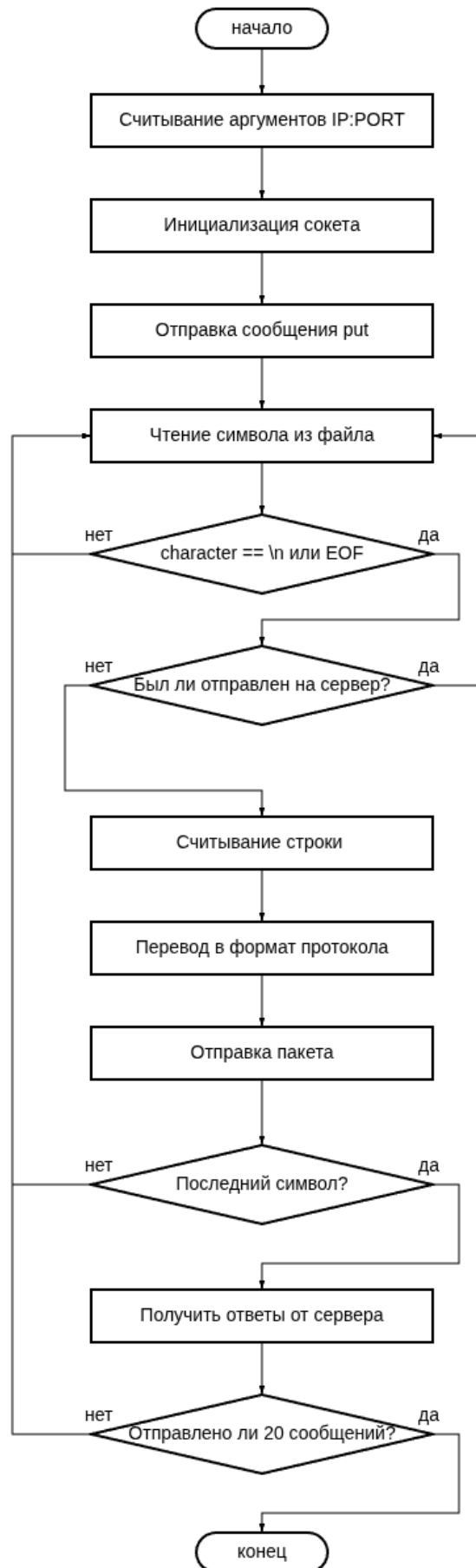


Рис. 3. Блок-схема UDP-клиента

Функция	Описание
<code>void send_request(int s, struct sockaddr_in* addr, unsigned char* datagram, uint32_t datagram_len)</code>	Отправляет датаграмм на адрес сервера с помощью sendto
<code>unsigned int recv_response(int s)</code>	Принимает от сервера список подтверждённых номеров сообщений через select с таймаутом и обновляет массив on_server
<code>void pack_message_data(...)</code>	Упаковывает поля сообщения в бинарный буфер в сетевом порядке байт (аналогично TCP-клиенту)
<code>unsigned int create_response(int s, struct sockaddr_in* addr, FILE* f)</code>	Читает строки из файла, пропускает уже подтверждённые сообщения и отправляет остальные пакеты серверу
<code>int main(int argc, char* argv[])</code>	Разбирает аргументы, создаёт UDP-сокет, запускает цикл отправки с повторными передачами до получения всех подтверждений

Таблица 3. Функция программы UDP-клиента

1. Описание алгоритма работы UDP-сервера

UDP-сервер при запуске создаёт один или несколько датаграммных сокетов, привязывая их к диапазону портов, переданному в аргументах командной строки. Для мультиплексирования входящих датаграмм используется функция `select`, которая ожидает активности сразу на всех сокетах с таймаутом одну секунду. При получении датаграммы сервер идентифицирует клиента по паре IP-адрес и порт, находя или создавая для него запись в массиве `clients`. Если сообщение с данным номером ещё не было принято от этого клиента, сервер декодирует пакет, записывает сообщение в файл `msg.txt` и добавляет номер в список принятых. В ответ на каждую датаграмму сервер отправляет клиенту список последних подтверждённых номеров сообщений (до 20 штук), что позволяет клиенту определить, какие пакеты можно не повторять. При получении сообщения «stop» сервер завершает цикл обработки, закрывает все сокеты и файл.

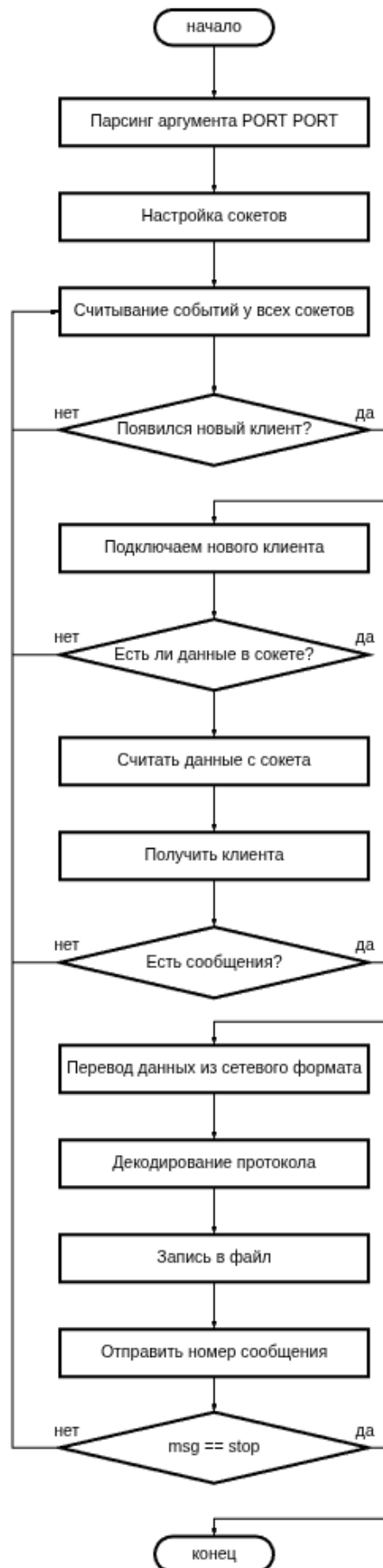


Рис. 4. Блок-схема UDP-сервера

Функция	Описание
<code>client_t* get_client(client_t* clients, struct sockaddr_in* addr)</code>	Ищет существующую запись клиента по адресу или создаёт новую в массиве clients
<code>int has_msg(client_t* c, uint32_t id)</code>	Проверяет, было ли сообщение с данным номером уже принято от данного клиента
<code>void add_msg(client_t* c, uint32_t id)</code>	Добавляет номер принятого сообщения в список клиента для последующей отправки подтверждения
<code>void send_response(socket_t s, client_t* c, struct sockaddr_in* addr)</code>	Формирует и отправляет клиенту датаграмм со списком последних подтверждённых номеров сообщений
<code>void write_msg(FILE* f, struct sockaddr_in* addr, ...)</code>	Форматирует и записывает принятое сообщение в файл msg.txt с адресом, портом и временными метками
<code>int main(int argc, char* argv[])</code>	Инициализирует сокеты для диапазона портов, запускает цикл select и обрабатывает входящие датаграммы от всех клиентов

Таблица 4. Функция программы UDP-сервера

Так же ниже на рисунках Рис. 5 и Рис. 6 приведены временные диаграммы работы сервера и клиента, и их взаимодействия.

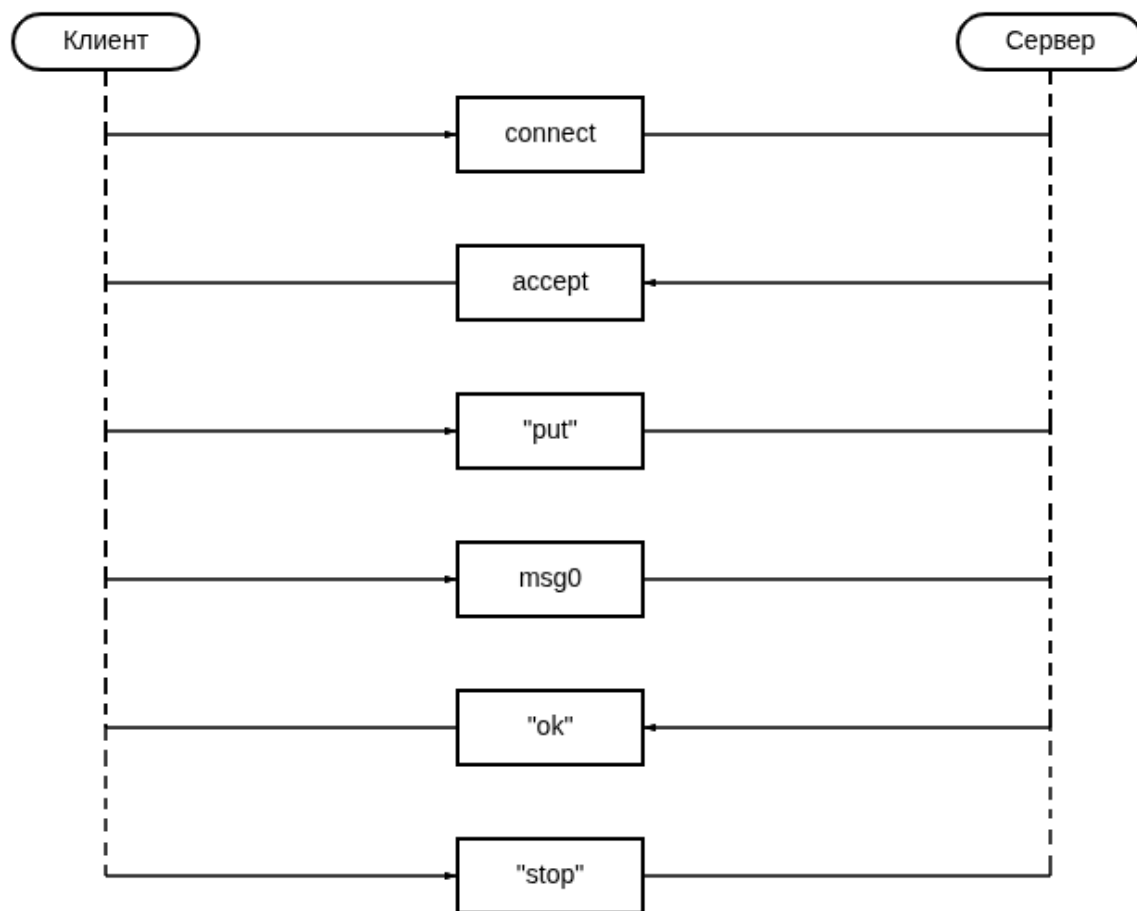


Рис. 5. Временная диаграмма работы tcpserver и tcpclient

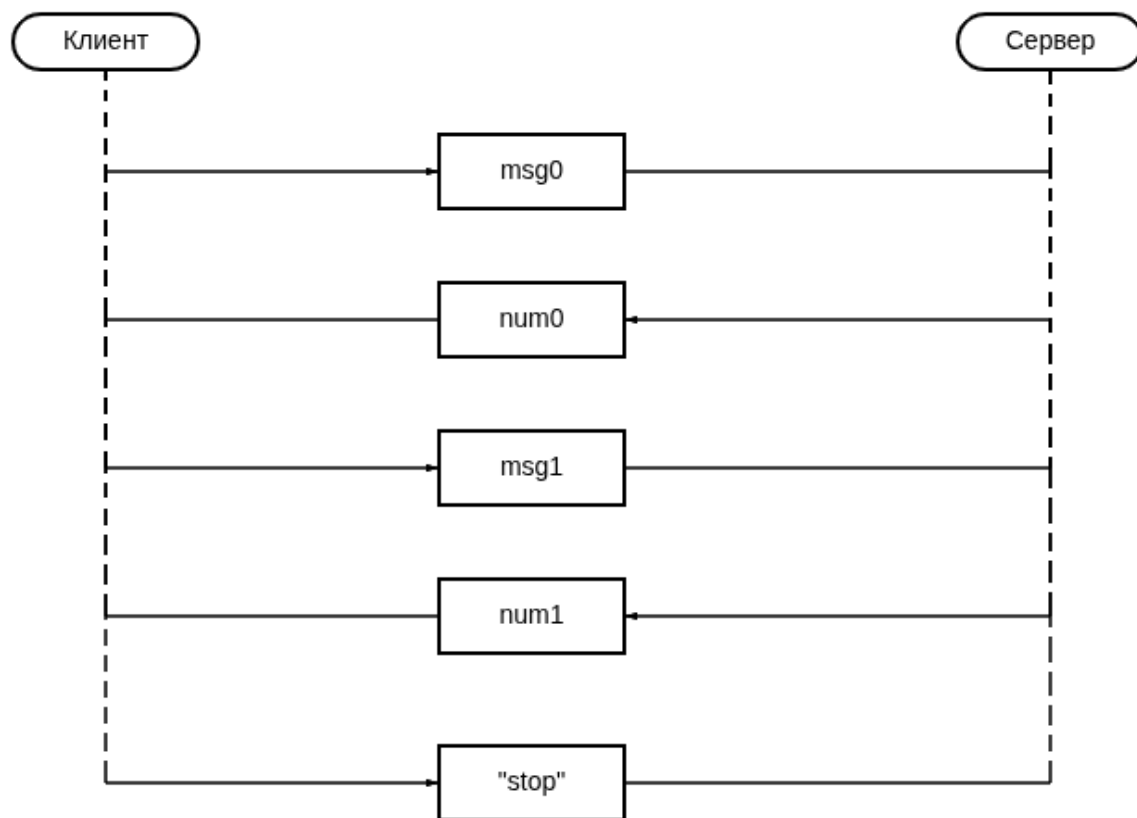


Рис. 6. Временная диаграмма работы udpserver и udpclient

4. Тестирование

Результаты тестирования приведены на рисунках ниже (Рис. 7 - Рис. 18), взяты первые три теста с учебной платформы ibks на программу. Изначальные тесты не работали и были исправлены.

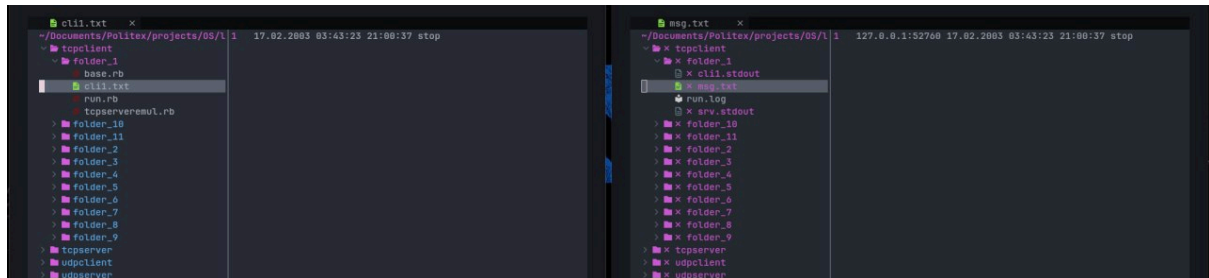


Рис. 7. Результат теста cli1.txt для программы tcpclient

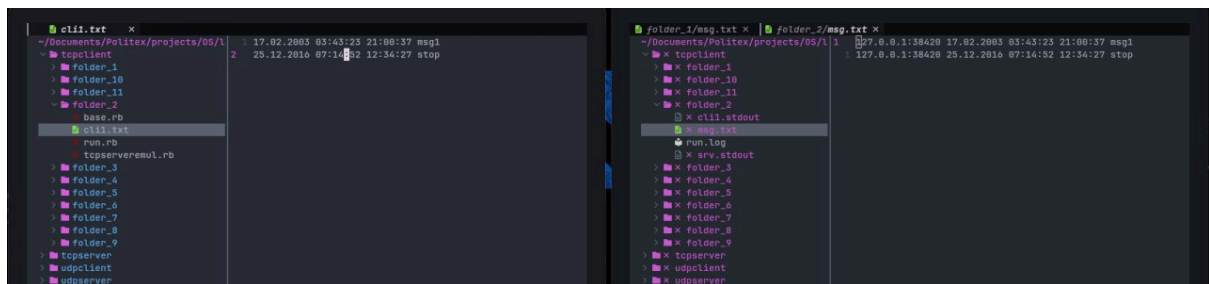


Рис. 8. Результат теста cli2.txt для программы tcpclient

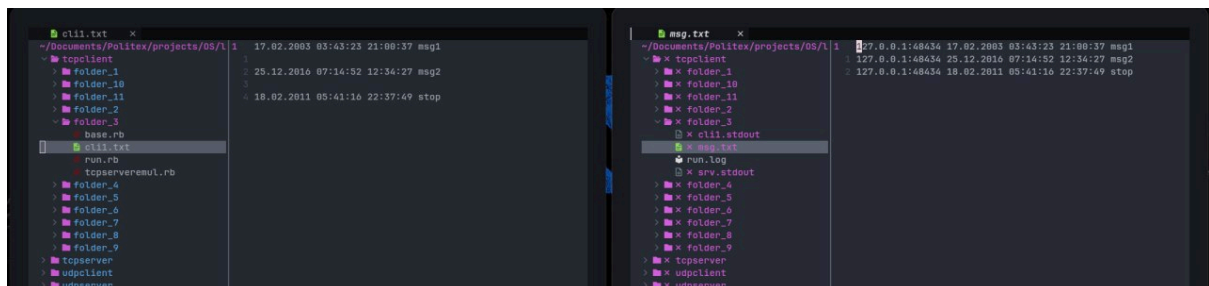


Рис. 9. Результат теста cli3.txt для программы tcpclient

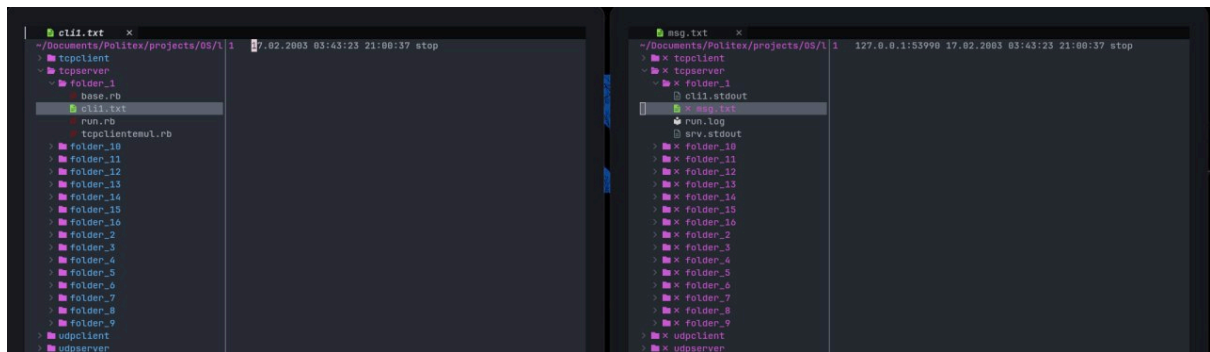


Рис. 10. Результат теста cli1.txt для программы tcpserver

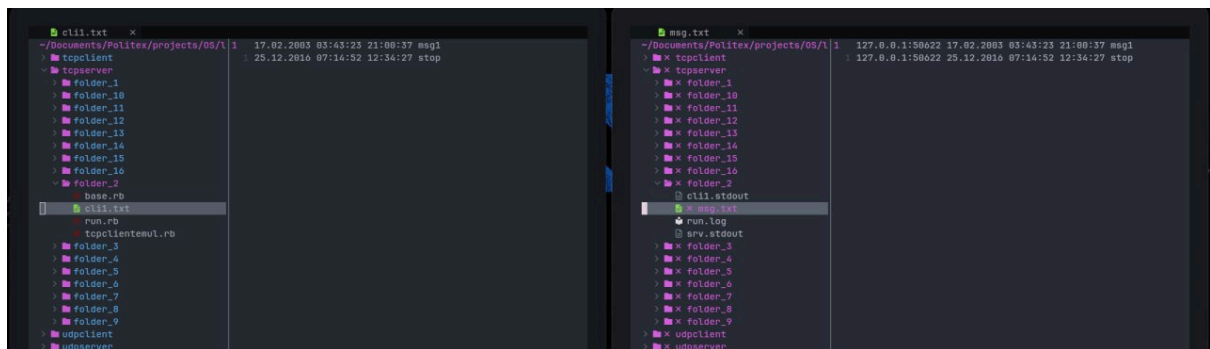


Рис. 11. Результат теста cli2.txt для программы tcpserver

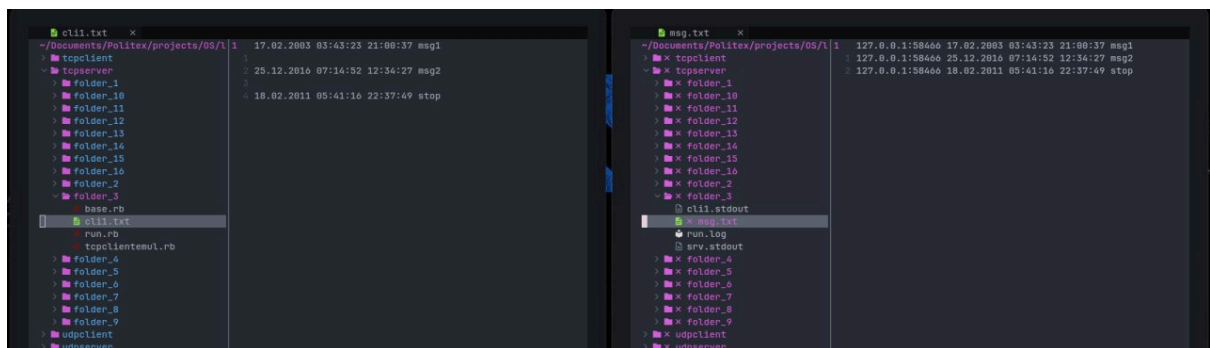


Рис. 12. Результат теста cli3.txt для программы tcpserver

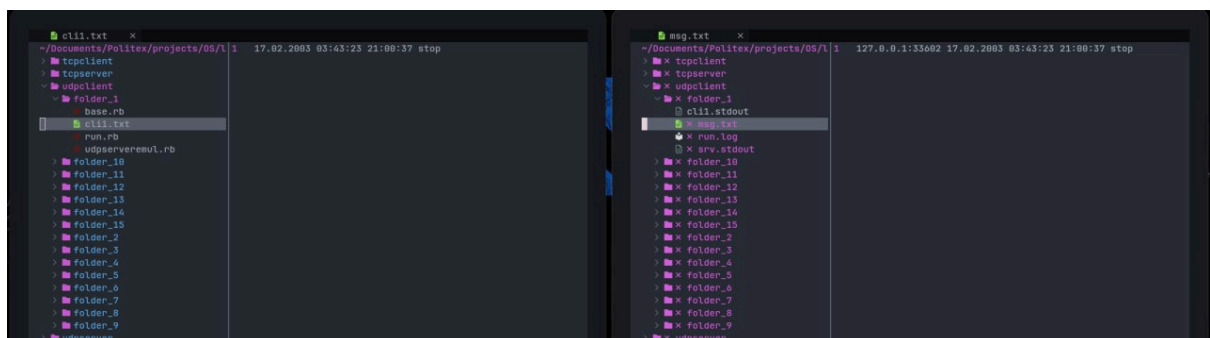


Рис. 13. Результат теста cli1.txt для программы udpclient

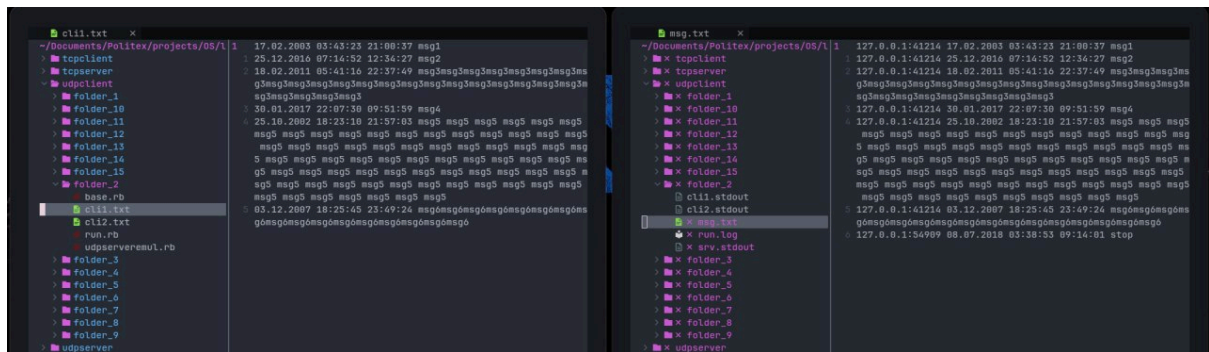


Рис. 14. Результат теста cli2.txt для программы udpclient

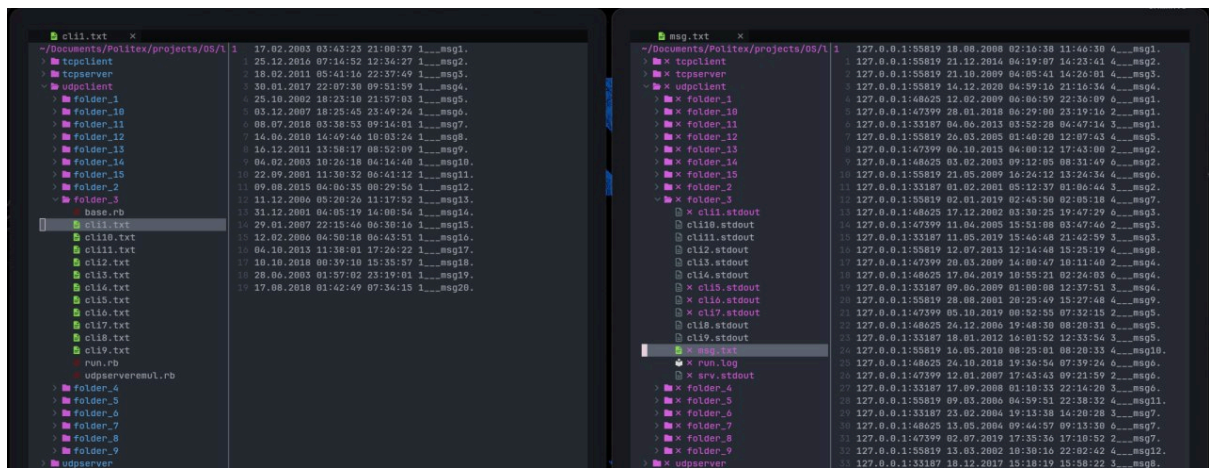


Рис. 15. Результат теста cli3.txt для программы udpclient

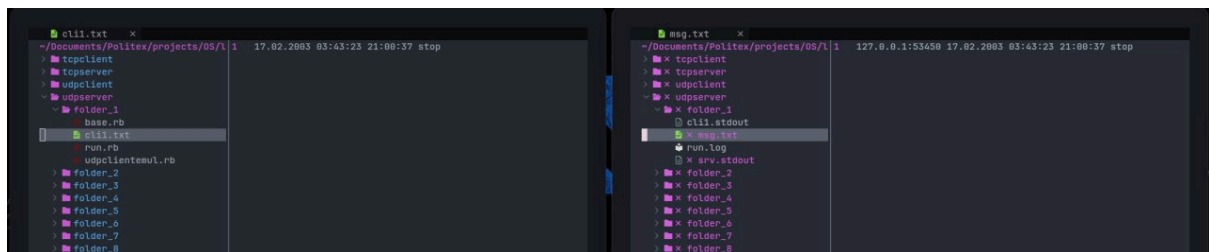


Рис. 16. Результат теста cli1.txt для программы udpserver

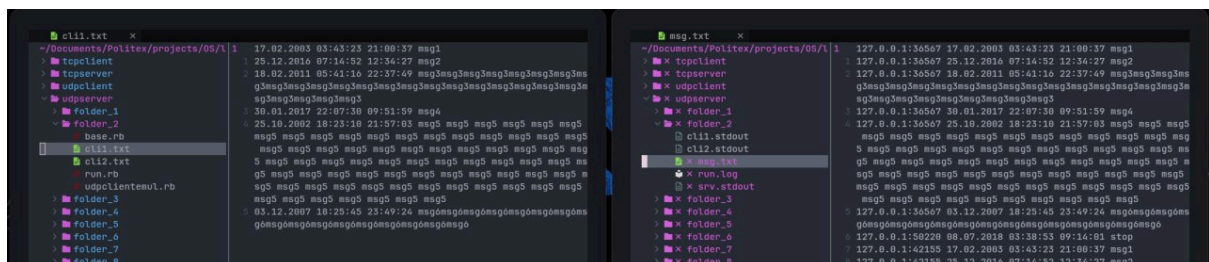


Рис. 17. Результат теста cli2.txt для программы udpserver

```
cl11.txt x
~/Documents/Politec/projects/05/1
tcpclient
tcpserver
udpclient
udpserver
folder_1
folder_2
folder_3
base.rb
cl11.txt
cl119.txt
cl111.txt
cl12.txt
cl13.txt
cl14.txt
cl15.txt
cl16.txt
cl17.txt
cl18.txt
cl19.txt
run.rb
udpclientemul.rb
folder_4
folder_5
folder_6
folder_7
folder_8

17.02.2003 03:43:23 21:00:37 cl11.msg1.msg1.msg1.msg1.msg1
msg1.msg1.msg1.msg1.msg1.msg1.msg1.msg1.msg1.msg1.msg1
msg1.msg1.msg1.msg1.msg1.msg1.msg1.msg1.msg1.msg1.msg1
1.msg1.msg1.msg1.msg1.msg1.msg1.msg1
25.12.2016 07:14:52 12:34:27 cl11.msg2.msg2.msg2.msg2
msg2.msg2.msg2.msg2.msg2.msg2.msg2.msg2.msg2.msg2
msg2.msg2.msg2.msg2.msg2.msg2.msg2.msg2.msg2.msg2
2.msg2.msg2.msg2.msg2.msg2.msg2.msg2
18.02.2011 05:41:16 22:37:49 cl11.msg3.msg3.msg3.msg3
msg3.msg3.msg3.msg3.msg3.msg3.msg3.msg3.msg3.msg3
msg3.msg3.msg3.msg3.msg3.msg3.msg3.msg3.msg3.msg3
msg3.msg3.msg3.msg3.msg3.msg3.msg3.msg3.msg3.msg3
3.msg3.msg3.msg3.msg3.msg3.msg3.msg3
30.01.2017 22:07:30 09:51:59 cl11.msg4.msg4.msg4.msg4
msg4.msg4.msg4.msg4.msg4.msg4.msg4.msg4.msg4.msg4
msg4.msg4.msg4.msg4.msg4.msg4.msg4.msg4.msg4.msg4
4.msg4.msg4.msg4.msg4.msg4.msg4.msg4
25.10.2002 10:23:10 21:57:03 cl11.msg5.msg5.msg5.msg5
msg5.msg5.msg5.msg5.msg5.msg5.msg5.msg5.msg5.msg5
msg5.msg5.msg5.msg5.msg5.msg5.msg5.msg5.msg5.msg5
5.msg5.msg5.msg5.msg5.msg5.msg5.msg5
03.12.2009 18:25:45 23:49:04 cl11.msg6.msg6.msg6.msg6
msg6.msg6.msg6.msg6.msg6.msg6.msg6.msg6.msg6.msg6
msg6.msg6.msg6.msg6.msg6.msg6.msg6.msg6.msg6.msg6
6.msg6.msg6.msg6.msg6.msg6.msg6.msg6
08.07.2018 03:38:53 09:14:01 cl11.msg7.msg7.msg7.msg7
msg7.msg7.msg7.msg7.msg7.msg7.msg7.msg7.msg7.msg7
msg7.msg7.msg7.msg7.msg7.msg7.msg7.msg7.msg7.msg7

msg.txt x
~/Documents/Politec/projects/05/1
tcpclient
tcpserver
udpclient
udpserver
folder_1
folder_2
folder_3
x
cl11.stdout
cl118.stdout
cl111.stdout
cl12.stdout
cl13.stdout
cl14.stdout
cl15.stdout
cl16.stdout
cl17.stdout
cl18.stdout
cl19.stdout
x
run.log
x
srv.stdout
x
folder_4
x
folder_5
x
folder_6
x
folder_7
x
folder_8

127.0.0.1:40875 28.01.2018 06:29:00 23:19:16 cl12.msg1
msg1.msg1.msg1.msg1.msg1.msg1.msg1.msg1.msg1.msg1
1.msg1.msg1.msg1.msg1.msg1.msg1.msg1.msg1.msg1.msg1
1.msg1.msg1.msg1.msg1.msg1.msg1.msg1
127.0.0.1:40875 06.10.2015 04:00:12 17:43:08 cl12.msg2
msg2.msg2.msg2.msg2.msg2.msg2.msg2.msg2.msg2.msg2
2.msg2.msg2.msg2.msg2.msg2.msg2.msg2.msg2.msg2.ms
g2.msg2.msg2.msg2.msg2.msg2.msg2.msg2
127.0.0.1:40875 11.04.2005 15:51:08 03:47:46 cl12.msg3
msg3.msg3.msg3.msg3.msg3.msg3.msg3.msg3.msg3.msg3
3.msg3.msg3.msg3.msg3.msg3.msg3.msg3.msg3.msg3.ms
g3.msg3.msg3.msg3.msg3.msg3.msg3.msg3.msg3.msg3
127.0.0.1:40875 29.03.2009 14:00:47 10:11:40 cl12.msg4
msg4.msg4.msg4.msg4.msg4.msg4.msg4.msg4.msg4.ms
g4.msg4.msg4.msg4.msg4.msg4.msg4.msg4.msg4.ms
g4.msg4.msg4.msg4.msg4.msg4.msg4.msg4
127.0.0.1:40875 08.10.2019 00:52:55 07:32:15 cl12.msg5
msg5.msg5.msg5.msg5.msg5.msg5.msg5.msg5.msg5.ms
g5.msg5.msg5.msg5.msg5.msg5.msg5.msg5.msg5.ms
g5.msg5.msg5.msg5.msg5.msg5.msg5.msg5
127.0.0.1:40875 12.01.2007 17:43:43 09:21:59 cl12.msg6
msg6.msg6.msg6.msg6.msg6.msg6.msg6.msg6.msg6.ms
g6.msg6.msg6.msg6.msg6.msg6.msg6.msg6.msg6.ms
g6.msg6.msg6.msg6.msg6.msg6.msg6.msg6
127.0.0.1:40875 02.07.2019 17:35:36 17:10:32 cl12.msg7
msg7.msg7.msg7.msg7.msg7.msg7.msg7.msg7.msg7.ms
g7.msg7.msg7.msg7.msg7.msg7.msg7.msg7.msg7.ms
g7.msg7.msg7.msg7.msg7.msg7.msg7.msg7
```

Рис. 18. Результат теста cl13.txt для программы udpserver

5. Выводы

В результате выполнения лабораторной работы был изучен программный интерфейс сетевых сокетов, получены практические навыки разработки клиент-серверных приложений на основе протоколов TCP и UDP, а также освоены методы организации надёжной передачи данных и обслуживания нескольких клиентов одновременно.

Приложения

Приложение А

tcp_client.c

```
#ifdef _WIN32
#define WIN32_LEAN_AND_MEAN
#include <windows.h>
#include <winsock2.h>
#include <ws2tcpip.h>
#pragma comment(lib, "ws2_32.lib")
#else
#include <sys/types.h>
#include <sys/socket.h>
#include <netdb.h>
#include <errno.h>

#include <arpa/inet.h>
#include <unistd.h>
#include <netinet/in.h>
#endif

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define DEBUG 1

int init()
{
#ifdef _WIN32
    WSADATA wsa_data;
    return (0 == WSStartup(MAKEWORD(2, 2), &wsa_data));
#else
    return 1;
#endif
}

void deinit()
```

```

{
#ifdef _WIN32
    WSACleanup();
#else

#endif
}

int sock_err(const char* function, int s)
{
    int err;
#ifdef _WIN32
    err = WSAGetLastError();
#else
    err = errno;
#endif
    fprintf(stderr, "%s: socket error: %d\n", function, err);
    return -1;
}

void s_close(int s)
{
#ifdef _WIN32
    closesocket(s);
#else
    close(s);
#endif
}

int send_request(int s, unsigned char* request, int size)
{
    // int size = strlen((const char*)request);

    int sent = 0;

#ifdef _WIN32
    int flags = 0;
#else
    int flags = MSG_NOSIGNAL;
#endif
}

```



```

while (sent < size)
{
    // Отправка очередного блока данных
    int res = send(s, request + sent, size - sent, flags);

    if (res < 0)
        return sock_err("send", s);

    sent += res;
#ifdef DEBUG
    printf("%d bytes sent. \n", sent);
#endif
}

return 0;
}

void pack_message_data(uint32_t msg_num, uint32_t date, uint32_t
time1,
                        uint32_t time2, char* message, unsigned char* buffer)
{
    int offset = 0;
    uint32_t net_value;

    // 1. Номер сообщения (4 байта)
    net_value = htonl(msg_num);
    memcpy(buffer + offset, &net_value, 4);
    offset += 4;

    // 2. Дата (4 байта)
    net_value = htonl(date);
    memcpy(buffer + offset, &net_value, 4);
    offset += 4;

    // 3. Первое время (4 байта)
    net_value = htonl(time1);
    memcpy(buffer + offset, &net_value, 4);
    offset += 4;

```

```

// 4. Второе время (4 байта)
net_value = htonl(time2);
memcpy(buffer + offset, &net_value, 4);
offset += 4;

// 5. Длина сообщения (4 байта)
uint32_t msg_len = strlen(message);
net_value = htonl(msg_len);
memcpy(buffer + offset, &net_value, 4);
offset += 4;

// 6. Текст сообщения (N байт)
memcpy(buffer + offset, message, msg_len);
}

int recv_ok(int s)
{
    char buf[2];
    int received = 0;

    while (received < 2)
    {
        int r = recv(s, buf + received, 2 - received, 0);
        if (r == 0)
        {
            // Сервер закрыл соединение – скорее всего stop
            return 1; // Не ошибка, просто завершение
        }
        if (r <= 0)
            return sock_err("recv", s);

        received += r;
    }

    if (buf[0] != 'o' || buf[1] != 'k')
    {
        printf("Protocol error\n");
        return -1;
    }
}

```

```

    return 0;
}

void addChar(char* s, char c)
{
    // Move pointer to the end
    while (*s++)
        ;

    // Append the new character
    *(s - 1) = c;

    // Add null terminator to mark new end
    *s = '\0';
}

int tok_numbers(char* src, const char* delim, bool is_reverse)
{
    char* saveptr;
    char* token;
    int final_date = 0;

    token = strtok_r(src, delim, &saveptr);

    if (is_reverse)
        final_date += atoi(token);
    else
        final_date += 10000 * atoi(token);

    token = strtok_r(NULL, delim, &saveptr);
    final_date += 100 * atoi(token);

    token = strtok_r(NULL, delim, &saveptr);

    if (is_reverse)
        final_date += 10000 * atoi(token);
    else
        final_date += atoi(token);

    return final_date;
}

```

```
}
```

```
unsigned int create_response(int s, FILE* f)
```

```
{
```

```
    int len_step = 256;
```

```
    unsigned int message_counter = 0;
```

```
    char* response = (char*)calloc(len_step, sizeof(char));
```

```
    if (!response)
```

```
    {
```

```
        printf("Memory allocation failed\n");
```

```
        return -1;
```

```
    }
```

```
    int character = '\\0';
```

```
    bool find_sym = false;
```

```
    int len_str = len_step;
```

```
    int buffer_size = 0;
```

```
    do
```

```
    {
```

```
        character = fgetc(f);
```

```
        if (character == '\\r')
```

```
            continue;
```

```
        // printf("%c", character);
```

```
        if ((character == '\\n' || character == EOF) && find_sym)
```

```
        {
```

```
            char* date;
```

```
            char* time1;
```

```
            char* time2;
```

```
            /* strncpy(msg, response + 29, strlen(response) - 29); */
```

```
            size_t resp_len = strlen(response);
```

```
            unsigned int msg_len = resp_len - 29;
```

```
            char* msg = (char*)malloc(msg_len + 1);
```

```

memcpy(msg, response + 29, msg_len);
msg[msg_len] = '\0';

char* saveptr;
date = strtok_r(response, " ", &saveptr);
time1 = strtok_r(NULL, " ", &saveptr);
time2 = strtok_r(NULL, " ", &saveptr);

unsigned int final_date = tok_numbers(date, ".", true);
unsigned int final_time1 = tok_numbers(time1, ":", false);
unsigned int final_time2 = tok_numbers(time2, ":", false);

#if DEBUG
    printf("%d | %d | %d | %d | %d | %s\n", message_counter,
        final_date,
            final_time1, final_time2, msg_len, msg);
#endif

uint32_t packet_size = 20 + msg_len;

unsigned char* output = (unsigned char*)malloc(packet_size);

pack_message_data(message_counter, final_date, final_time1,
    final_time2, msg, output);

send_request(s, output, packet_size);

recv_ok(s);

find_sym = false;

memset(response, 0, len_str);

if (strcmp(msg, "stop") != 0)
{
    #if DEBUG
        printf("stop\n");
    #endif
    message_counter++;
}

```

```

    buffer_size = 0;

    free(msg);
    free(output);

    if (character == EOF)
        break;
}

if (buffer_size >= len_str - 1)
{
    len_str += len_step;

    char* tmp_str = (char*)realloc(response, len_str *
sizeof(char));
    if (tmp_str == NULL)
    {
        printf("Memory reallocation failed\n");
        free(response);
        return -1;
    }

    response = tmp_str;
}

if (!find_sym && character >= '0' && character <= '9')
{
    find_sym = true;
}

if (find_sym)
{
    addChar(response, character);
    buffer_size++;
}
} while (1);

free(response);
response = NULL;

```

```

#ifdef DEBUG
    printf("\n");
    printf("end of function\n");
#endif

    return message_counter;
}

int main(int argc, char* argv[])
{
    const char* address;
    char filename[255];
    int port;
    if (argv[1])
    {
        char* token = strtok(argv[1], ":");
        address = token;
        token = strtok(NULL, ":");
        port = atoi(token);
#ifdef DEBUG
        printf("%s %d\n", address, port);
#endif
    }

    if (argv[2])
    {
        int len = strlen(argv[2]);
        strncpy(filename, argv[2], len);
        filename[len] = '\0';
#ifdef DEBUG
        printf("%s\n", filename);
#endif
    }

    int s;
    struct sockaddr_in addr;

    FILE* f;

```

```

f = fopen(filename, "r");

init();

s = socket(AF_INET, SOCK_STREAM, 0);
if (s < 0)
    return sock_err("socket", s);

memset(&addr, 0, sizeof(addr));
addr.sin_family = AF_INET;
addr.sin_port = htons(port);
addr.sin_addr.s_addr = inet_addr(address);

int attempt;
for (attempt = 0; attempt < 10; attempt++)
{
    if (connect(s, (struct sockaddr*)&addr, sizeof(addr)) == 0)
    {
        printf("Connected\n");
        break;
    }

    printf("Connect attempt %d failed\n", attempt + 1);

    usleep(100000); // 100 ms
}

if (attempt == 10)
{
    printf("Unable to connect after 10 attempts\n");
    s_close(s);
    return 0;
}

send_request(s, (unsigned char*)"put", 3);

unsigned int count = create_response(s, f);
if (count < 0)
{
    printf("Error in client\n");
}

```



```

    }

    #if DEBUG
        printf("messages count: %d\n", count);
    #endif

    // for (unsigned int i = 0; i < count; i++)
    // {
    //     recv_ok(s);
    // }

    fclose(f);

    s_close(s);

    deinit();

    return 0;
}

```

tcpserver.c

```

#ifdef _WIN32
#define WIN32_LEAN_AND_MEAN
#include <winsock2.h>
#include <ws2tcpip.h>
#include <windows.h>
#pragma comment(lib, "ws2_32.lib")
#define poll WSAPoll
#else
#include <sys/socket.h>
#include <arpa/inet.h>
#include <sys/poll.h>
#include <unistd.h>
#include <fcntl.h>
#endif

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdint.h>

```

```

#define MAX_CLIENTS 10
#define BUFFER_SIZE 8192

#ifdef _WIN32
typedef SOCKET socket_t;
#else
typedef int socket_t;
#endif

typedef struct
{
    socket_t sock;
    int active;
    int ready;
    unsigned char* buf; // динамический буфер
    int len;
    int capacity;
    struct sockaddr_in addr;
} client_t;

void close_socket(socket_t s)
{
#ifdef _WIN32
    closesocket(s);
#else
    close(s);
#endif
}

void set_nonblocking(socket_t s)
{
#ifdef _WIN32
    u_long mode = 1;
    ioctlsocket(s, FIONBIO, &mode);
#else
    fcntl(s, F_SETFL, O_NONBLOCK);
#endif
}

uint32_t read_u32(unsigned char* p)

```

```

{
    uint32_t v;
    memcpy(&v, p, 4);
    return ntohl(v);
}

void write_msg(FILE* f, struct sockaddr_in* addr, uint32_t date,
uint32_t t1,
    uint32_t t2, char* msg)
{
    int yyyy = date / 10000;
    int mm = (date / 100) % 100;
    int dd = date % 100;

    int h1 = t1 / 10000;
    int m1 = (t1 / 100) % 100;
    int s1 = t1 % 100;

    int h2 = t2 / 10000;
    int m2 = (t2 / 100) % 100;
    int s2 = t2 % 100;

    fprintf(f, "%s:%d %02d.%02d.%04d %02d:%02d:%02d %02d:%02d:%02d
%s\n",
        inet_ntoa(addr->sin_addr), ntohs(addr->sin_port), dd, mm,
yyyy, h1,
        m1, s1, h2, m2, s2, msg);

    fflush(f);
}

int main(int argc, char* argv[])
{
#ifdef _WIN32
    WSADATA w;
    WSStartup(MAKEWORD(2, 2), &w);
#endif

    if (argc < 2)

```

```

{
    printf("usage: tcpserver <port>\n");
    return 1;
}

int port = atoi(argv[1]);

FILE* f = fopen("msg.txt", "a");
if (!f)
{
    printf("cannot open msg.txt\n");
    return 1;
}

socket_t ls = socket(AF_INET, SOCK_STREAM, 0);

if (ls < 0)
{
    printf("socket error\n");
    return 1;
}

struct sockaddr_in addr;

memset(&addr, 0, sizeof(addr));
addr.sin_family = AF_INET;
addr.sin_port = htons(port);
addr.sin_addr.s_addr = INADDR_ANY;

if (bind(ls, (struct sockaddr*)&addr, sizeof(addr)) < 0)
{
    printf("bind error\n");
    return 1;
}

if (listen(ls, 10) < 0)
{
    printf("listen error\n");
    return 1;
}

```

```

set_nonblocking(ls);

client_t clients[MAX_CLIENTS];
struct pollfd pfd[MAX_CLIENTS + 1];

int i;

for (i = 0; i < MAX_CLIENTS; i++)
{
    clients[i].active = 0;
    pfd[i].fd = -1;
}

pfd[MAX_CLIENTS].fd = ls;
pfd[MAX_CLIENTS].events = POLLIN;

int stop = 0;

while (!stop)
{
    int n = poll(pfd, MAX_CLIENTS + 1, 1000);

    if (n <= 0)
        continue;

    if (pfd[MAX_CLIENTS].revents & POLLIN)
    {
        struct sockaddr_in caddr;
#ifdef _WIN32
        int len = sizeof(caddr);
#else
        socklen_t len = sizeof(caddr);
#endif

        socket_t cs = accept(ls, (struct sockaddr*)&caddr, &len);

        if (cs >= 0)
        {

```

```

set_nonblocking(cs);

for (i = 0; i < MAX_CLIENTS; i++)
    if (!clients[i].active)
    {
        clients[i].active = 1;
        clients[i].sock = cs;
        clients[i].ready = 0;
        clients[i].len = 0;
        clients[i].capacity = BUFFER_SIZE;
        clients[i].buf =
            (unsigned char*)malloc(clients[i].capacity);
        clients[i].addr = caddr;

        pfd[i].fd = cs;
        pfd[i].events = POLLIN;
        break;
    }

    if (i == MAX_CLIENTS)
        close_socket(cs);
}

for (i = 0; i < MAX_CLIENTS; i++)
{

    if (!clients[i].active)
        continue;

    if (pfd[i].revents & (POLLERR | POLLHUP))
    {
        free(clients[i].buf);
        close_socket(clients[i].sock);
        clients[i].active = 0;
        pfd[i].fd = -1;
        continue;
    }

    if (pfd[i].revents & POLLIN)

```

```

{

    // расширяем буфер если заполнен
    if (clients[i].len == clients[i].capacity)
    {
        clients[i].capacity *= 2;
        clients[i].buf = (unsigned char*)realloc(
            clients[i].buf, clients[i].capacity);
    }

    int r = recv(clients[i].sock,
        (char*)clients[i].buf + clients[i].len,
        clients[i].capacity - clients[i].len, 0);

    if (r <= 0)
    {
        free(clients[i].buf);
        close_socket(clients[i].sock);
        clients[i].active = 0;
        pfd[i].fd = -1;
        continue;
    }

    clients[i].len += r;

    if (!clients[i].ready)
    {
        if (clients[i].len >= 3)
        {
            if (memcmp(clients[i].buf, "put", 3) != 0)
            {
                free(clients[i].buf);
                close_socket(clients[i].sock);
                clients[i].active = 0;
                pfd[i].fd = -1;
                continue;
            }

            memmove(clients[i].buf, clients[i].buf + 3,
                clients[i].len - 3);
        }
    }
}

```

```

        clients[i].len -= 3;
        clients[i].ready = 1;
    }
}

while (clients[i].ready && clients[i].len >= 20)
{

    unsigned char* p = clients[i].buf;

    uint32_t msgnum = read_u32(p);
    uint32_t date = read_u32(p + 4);
    uint32_t t1 = read_u32(p + 8);
    uint32_t t2 = read_u32(p + 12);
    uint32_t mlen = read_u32(p + 16);

    // гарантируем что буфер влезет
    if (clients[i].capacity < 20 + mlen)
    {
        while (clients[i].capacity < 20 + mlen)
            clients[i].capacity *= 2;

        clients[i].buf = (unsigned char*)realloc(
            clients[i].buf, clients[i].capacity);
    }

    if (clients[i].len < 20 + mlen)
        break;

    char* msg = (char*)malloc(mlen + 1);
    memcpy(msg, p + 20, mlen);
    msg[mlen] = 0;

    write_msg(f, &clients[i].addr, date, t1, t2, msg);

    send(clients[i].sock, "ok", 2, 0);

    if (strcmp(msg, "stop") == 0)
        stop = 1;
}

```



```

        free(msg);

        memmove(clients[i].buf, clients[i].buf + 20 + mlen,
                clients[i].len - (20 + mlen));

        clients[i].len -= 20 + mlen;
    }
}

for (i = 0; i < MAX_CLIENTS; i++)
    if (clients[i].active)
    {
        free(clients[i].buf);
        close_socket(clients[i].sock);
    }

close_socket(ls);

fclose(f);

#ifdef _WIN32
    WSACleanup();
#endif

    return 0;
}

udpclient.c
#define _CRT_SECURE_NO_WARNINGS
#ifdef _WIN32
#define WIN32_LEAN_AND_MEAN
#include <windows.h>
#include <winsock2.h>
#include <ws2tcpip.h>
#pragma comment(lib, "ws2_32.lib")

typedef int socklen_t;

// правильный аналог strtok_r для Windows

```

```

#define strtok_r(str, delim, saveptr) strtok_s((str), (delim),
(saveptr))

#define bool int
#define true 1
#define false 0

#else
#include <sys/types.h>
#include <sys/socket.h>
#include <sys/time.h>
#include <sys/select.h>
#include <netdb.h>
#include <errno.h>
#include <unistd.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <stdbool.h>
#endif

#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <stdint.h>

#define DEBUG 1
#define MAX_MSG 100000

int on_server[MAX_MSG];

int init()
{
#ifdef _WIN32
    WSADATA wsa_data;
    return (0 == WSStartup(MAKEWORD(2, 2), &wsa_data));
#else
    return 1;
#endif
}

```

```

void deinit()
{
#ifdef _WIN32
    WSACleanup();
#endif
}

int sock_err(const char* function, int s)
{
    int err;
#ifdef _WIN32
    err = WSAGetLastError();
#else
    err = errno;
#endif
    fprintf(stderr, "%s: socket error: %d\n", function, err);
    return -1;
}

void s_close(int s)
{
#ifdef _WIN32
    closesocket(s);
#else
    close(s);
#endif
}

void send_request(int s, struct sockaddr_in* addr, unsigned char*
datagram,
                  uint32_t datagram_len)
{
#ifdef _WIN32
    int flags = 0;
#else
    int flags = MSG_NOSIGNAL;
#endif

    int res = sendto(s, (const char*)datagram, datagram_len, flags,
                     (struct sockaddr*)addr, sizeof(struct sockaddr_in));

```

```

    if (res <= 0)
        sock_err("sendto", s);
}

unsigned int recv_response(int s)
{
    char datagram[1024];
    struct timeval tv = {0, 100 * 1000};

    fd_set fds;
    FD_ZERO(&fds);
    FD_SET(s, &fds);

    int res = select(s + 1, &fds, 0, 0, &tv);

    if (res > 0)
    {
        struct sockaddr_in addr;
        socklen_t addrlen = sizeof(addr);

        int received = recvfrom(s, datagram, sizeof(datagram), 0,
                                (struct sockaddr*)&addr, &addrlen);

        uint32_t* nums = (uint32_t*)datagram;
        int count = received / 4;

        for (int i = 0; i < count; i++)
        {
            uint32_t num = ntohl(nums[i]);
            #if DEBUG
                printf("%u ", num);
            #endif
            if (num < MAX_MSG)
                on_server[num] = 1;
        }

        printf("\n");

        if (received <= 0)

```

```

    {
        sock_err("recvfrom", s);
        return 0;
    }

    return 1;
}
else if (res == 0)
{
    return 0;
}
else
{
    sock_err("select", s);
    return 0;
}
}

void pack_message_data(uint32_t msg_num, uint32_t date, uint32_t
time1,
                        uint32_t time2, char* message, unsigned char* buffer)
{
    int offset = 0;
    uint32_t net_value;

    net_value = htonl(msg_num);
    memcpy(buffer + offset, &net_value, 4);
    offset += 4;

    net_value = htonl(date);
    memcpy(buffer + offset, &net_value, 4);
    offset += 4;

    net_value = htonl(time1);
    memcpy(buffer + offset, &net_value, 4);
    offset += 4;

    net_value = htonl(time2);
    memcpy(buffer + offset, &net_value, 4);
    offset += 4;
}

```

```

uint32_t msg_len = strlen(message);
net_value = htonl(msg_len);
memcpy(buffer + offset, &net_value, 4);
offset += 4;

memcpy(buffer + offset, message, msg_len);
}

void addChar(char* s, char c)
{
    while (*s++)
        ;

    *(s - 1) = c;
    *s = '\0';
}

int tok_numbers(char* src, const char* delim, bool is_reverse)
{
    char* saveptr;
    char* token;
    int final_date = 0;

    token = strtok_r(src, delim, &saveptr);

    if (is_reverse)
        final_date += atoi(token);
    else
        final_date += 10000 * atoi(token);

    token = strtok_r(NULL, delim, &saveptr);
    final_date += 100 * atoi(token);

    token = strtok_r(NULL, delim, &saveptr);

    if (is_reverse)
        final_date += 10000 * atoi(token);
    else
        final_date += atoi(token);
}

```

```

    return final_date;
}

unsigned int create_response(int s, struct sockaddr_in* addr,
FILE* f)
{
    int len_step = 256;
    unsigned int message_counter = 0;

    char* response = (char*)calloc(len_step, sizeof(char));
    if (!response)
        return -1;

    int character;
    bool find_sym = false;
    int len_str = len_step;
    int buffer_size = 0;

    while (1)
    {
        character = fgetc(f);

        if (character == '\r')
            continue;

        if ((character == '\n' || character == EOF) && find_sym)
        {
            if (on_server[message_counter])
            {
                message_counter++;
                continue;
            }

            char* date;
            char* time1;
            char* time2;

            size_t resp_len = strlen(response);
            unsigned int msg_len = resp_len - 29;

```

```

char* msg = (char*)malloc(msg_len + 1);

memcpy(msg, response + 29, msg_len);
msg[msg_len] = '\0';

char* saveptr;
date = strtok_r(response, " ", &saveptr);
time1 = strtok_r(NULL, " ", &saveptr);
time2 = strtok_r(NULL, " ", &saveptr);

unsigned int final_date = tok_numbers(date, ".", true);
unsigned int final_time1 = tok_numbers(time1, ":", false);
unsigned int final_time2 = tok_numbers(time2, ":", false);

#if DEBUG
    printf("%d | %d | %d | %d | %d | %s\n", message_counter,
final_date,
        final_time1, final_time2, msg_len, msg);
#endif

uint32_t packet_size = 20 + msg_len;
unsigned char* output = (unsigned char*)malloc(packet_size);

pack_message_data(message_counter, final_date, final_time1,
    final_time2, msg, output);

send_request(s, addr, output, packet_size);

find_sym = false;
memset(response, 0, len_str);

message_counter++;
buffer_size = 0;

free(msg);
free(output);

if (character == EOF)
    break;

```



```

    }

    if (buffer_size >= len_str - 1)
    {
        len_str += len_step;
        char* tmp = (char*)realloc(response, len_str);
        if (!tmp)
        {
            free(response);
            return -1;
        }
        response = tmp;
    }

    if (!find_sym && character >= '0' && character <= '9')
        find_sym = true;

    if (find_sym)
    {
        addChar(response, character);
        buffer_size++;
    }
}

free(response);
return message_counter;
}

int main(int argc, char* argv[])
{
    const char* address;
    char filename[255];
    int port;

    if (argv[1])
    {
        char* token = strtok(argv[1], ":");
        address = token;
        token = strtok(NULL, ":");
        port = atoi(token);
    }
}

```

```

}

if (argv[2])
{
    int len = strlen(argv[2]);
    strncpy(filename, argv[2], len);
    filename[len] = '\0';
}

memset(on_server, 0, sizeof(on_server));

FILE* f = fopen(filename, "r");

int s;
struct sockaddr_in addr;

init();

s = socket(AF_INET, SOCK_DGRAM, 0);
if (s < 0)
    return sock_err("socket", s);

memset(&addr, 0, sizeof(addr));
addr.sin_family = AF_INET;
addr.sin_port = htons(port);
addr.sin_addr.s_addr = inet_addr(address);

int count = create_response(s, &addr, f);

int confirmed = 0;

while (1)
{
    fseek(f, 0, SEEK_SET);

    create_response(s, &addr, f);

    while (recv_response(s))
    {
    }
}

```

```

    confirmed = 0;
    for (int i = 0; i < count; i++)
        if (on_server[i])
            confirmed++;

    if (count < 20)
    {
        if (confirmed >= count)
            break;
    }
    else
    {
        if (confirmed >= 20)
            break;
    }
}

fclose(f);
s_close(s);
deinit();

return 0;
}

```

udpserver.c

```

#include <sys/socket.h>
#include <arpa/inet.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/select.h>

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdint.h>
#include <time.h>

#define MAX_CLIENTS 100
#define MAX_PORTS 32
#define MAX_MSG_PER_CLIENT 1000

```

```

#define BUFFER_SIZE 65535

typedef int socket_t;

typedef struct
{
    struct sockaddr_in addr;
    uint32_t msgs[MAX_MSG_PER_CLIENT];
    int msg_count;
    time_t last_tm;
    int active;
} client_t;

void set_nonblocking(socket_t s) { fcntl(s, F_SETFL,
O_NONBLOCK); }

uint32_t read_u32(unsigned char* p)
{
    uint32_t v;
    memcpy(&v, p, 4);
    return ntohl(v);
}

void write_msg(FILE* f, struct sockaddr_in* addr, uint32_t date,
uint32_t t1,
            uint32_t t2, char* msg)
{
    int yyyy = date / 10000;
    int mm = (date / 100) % 100;
    int dd = date % 100;

    int h1 = t1 / 10000;
    int m1 = (t1 / 100) % 100;
    int s1 = t1 % 100;

    int h2 = t2 / 10000;
    int m2 = (t2 / 100) % 100;
    int s2 = t2 % 100;

    fprintf(f, "%s:%d %02d.%02d.%04d %02d:%02d:%02d %02d:%02d:%02d

```

```

%s\n",
    inet_ntoa(addr->sin_addr), ntohs(addr->sin_port), dd, mm,
    yyyy, h1,
    m1, s1, h2, m2, s2, msg);

    fflush(f);
}

int same_client(struct sockaddr_in* a, struct sockaddr_in* b)
{
    return a->sin_addr.s_addr == b->sin_addr.s_addr &&
        a->sin_port == b->sin_port;
}

client_t* get_client(client_t* clients, struct sockaddr_in* addr)
{
    for (int i = 0; i < MAX_CLIENTS; i++)
        if (clients[i].active && same_client(&clients[i].addr, addr))
            return &clients[i];

    for (int i = 0; i < MAX_CLIENTS; i++)
        if (!clients[i].active)
        {
            clients[i].active = 1;
            clients[i].addr = *addr;
            clients[i].msg_count = 0;
            clients[i].last_tm = time(NULL);

            printf("New client %s:%d\n", inet_ntoa(addr->sin_addr),
                ntohs(addr->sin_port));

            return &clients[i];
        }

    return NULL;
}

int has_msg(client_t* c, uint32_t id)
{
    for (int i = 0; i < c->msg_count; i++)

```

```

        if (c->msgs[i] == id)
            return 1;
    return 0;
}

void add_msg(client_t* c, uint32_t id)
{
    if (c->msg_count < MAX_MSG_PER_CLIENT)
        c->msgs[c->msg_count++] = id;
}

void send_response(socket_t s, client_t* c, struct sockaddr_in*
addr)
{
    uint32_t buf[20];
    int count = 0;

    for (int i = c->msg_count - 1; i >= 0 && count < 20; i--)
        buf[count++] = htonl(c->msgs[i]);

    sendto(s, buf, count * 4, 0, (struct sockaddr*)addr,
sizeof(*addr));
}

int main(int argc, char* argv[])
{
    if (argc < 2)
    {
        printf("usage: udpserve port_from port_to\n");
        return 1;
    }

    int port_from = atoi(argv[1]);
    int port_to = (argc > 2) ? atoi(argv[2]) : port_from;

    socket_t socks[MAX_PORTS];
    int sock_count = 0;

    FILE* f = fopen("msg.txt", "a");

```

```

client_t clients[MAX_CLIENTS];
memset(clients, 0, sizeof(clients));

for (int p = port_from; p <= port_to; p++)
{
    socket_t s = socket(AF_INET, SOCK_DGRAM, 0);

    struct sockaddr_in addr;

    memset(&addr, 0, sizeof(addr));
    addr.sin_family = AF_INET;
    addr.sin_port = htons(p);
    addr.sin_addr.s_addr = INADDR_ANY;

    if (bind(s, (struct sockaddr*)&addr, sizeof(addr)) < 0)
    {
        printf("bind error\n");
        return 1;
    }

    set_nonblocking(s);

    socks[sock_count++] = s;

    printf("Listening on port %d\n", p);
}

int stop = 0;

while (!stop)
{
    fd_set rfd;

    FD_ZERO(&rfd);

    int maxfd = 0;

    for (int i = 0; i < sock_count; i++)
    {
        FD_SET(socks[i], &rfd);
    }
}

```

```

    if (socks[i] > maxfd)
        maxfd = socks[i];
}

struct timeval tv;
tv.tv_sec = 1;
tv.tv_usec = 0;

int r = select(maxfd + 1, &rfd, NULL, NULL, &tv);

if (r <= 0)
    continue;

for (int i = 0; i < sock_count; i++)
{
    if (!FD_ISSET(socks[i], &rfd))
        continue;

    unsigned char buf[BUFFER_SIZE];

    struct sockaddr_in cli;
    socklen_t len = sizeof(cli);

    int n = recvfrom(socks[i], buf, BUFFER_SIZE, 0,
                     (struct sockaddr*)&cli, &len);

    if (n <= 0)
        continue;

    uint32_t msgnum = read_u32(buf);

    client_t* c = get_client(clients, &cli);

    if (!c)
        continue;

    if (!has_msg(c, msgnum))
    {
        uint32_t date = read_u32(buf + 4);
        uint32_t t1 = read_u32(buf + 8);
    }
}

```



```

uint32_t t2 = read_u32(buf + 12);
uint32_t mlen = read_u32(buf + 16);

if (20 + mlen <= n)
{
    char* msg = (char*)malloc(mlen + 1);
    if (!msg)
        continue;

    memcpy(msg, buf + 20, mlen);
    msg[mlen] = 0;

    write_msg(f, &cli, date, t1, t2, msg);

    add_msg(c, msgnum);

    if (strcmp(msg, "stop") == 0)
        stop = 1;

    free(msg);
}
}

c->last_tm = time(NULL);

send_response(socks[i], c, &cli);
}
}

for (int i = 0; i < sock_count; i++)
    close(socks[i]);

fclose(f);

return 0;
}

```